Glados

Lexing

We used MegaParsec as our lexer tool.
It is widely used, as an improved fork of Parsec.
It has incredible error showing capabilities and great monadic implementation.
The output of the lexer is a LISP symbolic expression.

Parsing

After lexing is done, we simply parse LISP Symbolic expression (it's surprisingly easy) The parsing outputs an AST, designed to include :

- Define calls
- Lambda
- Values of type Int or Boolean
- If conditionals
- Named function calls (including builtins and user defined)
- Unnamed lambda calls

Execution

The execution is managed with full error feedback and recursivity implementation.

Testing

We've used unit test using hUnit and functional test using Python to cover as much possibilities as possible.

CI/CD

The CI test for compilation, coding-style compliance and full unit and functional test coverage.
The CD is used to create a release on the base and mirror repo.
It is triggered whenever a tag starting in v (example v1.0.0) is pushed and the CI is validated.

Errors

Errors that arises from any part of the process, from reading stdin to execution are reported to the user and the 84 exit code is returned.